



同濟大學
TONGJI UNIVERSITY

计算机系统结构课程实验 总结报告

实验题目：简单的流水线 CPU 设计与性能分析

学号：2352628

姓名：韦畅

指导教师：秦国锋

日期：2025.11.18

一、 实验环境部署与硬件配置说明

- 开发环境: Vivado 2016.2
- 仿真工具: Vivado 内置仿真器
- 汇编工具: MARS 4.5 (MIPS Assembler and Runtime Simulator)
- 开发板: Xilinx NEXYS4 DDR Artix-7 FPGA 开发板

二、 实验的总体结构

2.1 流水线 CPU 整体架构

本实验设计了一个基于 MIPS 指令集的五段静态流水线 CPU，整体架构分为五个相互独立但又协同工作的流水段：IF（指令获取）阶段、ID（指令译码）阶段、EX（执行）阶段、MEM（访存）阶段和 WB（写回）阶段。

2.2 IF（指令获取）阶段

- **主要功能:** 从指令存储器中读取指令
- **核心组件:**
 - PC 寄存器: 保存当前指令地址
 - 指令存储器 (IMEM): 存储程序指令
 - 地址计算单元: 计算下一条指令地址
- **关键操作:**
 - PC $\rightarrow$ IMEM: 根据 PC 值读取指令
 - PC+4 $\rightarrow$ NPC: 计算下一条顺序指令地址
 - 分支/跳转地址选择: 根据控制信号选择下一条指令地址

2.3 ID（指令译码）阶段

- **主要功能:** 解析指令并生成控制信号
- **核心组件:**
 - 控制器 (Controller): 解析指令操作码和功能码
 - 寄存器堆 (Regfile): 读取源操作数寄存器
 - 立即数扩展单元: 对立即数进行符号/零扩展

- **关键操作：**
 - 指令字段解析：opcode、rs、rt、rd、immediate 等
 - 寄存器读取：同时读取两个源寄存器
 - 控制信号生成：ALU 操作、存储器访问、写回控制等

2.4 EX（执行）阶段

- **主要功能：**执行算术逻辑运算
- **核心组件：**
 - ALU（算术逻辑单元）：执行各种运算
 - 多路选择器：选择 ALU 输入操作数
- **关键操作：**
 - 算术运算：加、减、乘、除等
 - 逻辑运算：与、或、非、异或等
 - 比较运算：大小比较、相等判断
 - 地址计算：分支目标地址、存储器地址计算

2.5 MEM（访存）阶段

- **主要功能：**数据存储器访问
- **核心组件：**
 - 数据存储器 (DMEM)：存储数据
 - 地址对齐单元：处理非对齐访问
- **关键操作：**
 - 加载操作：从存储器读取数据到寄存器
 - 存储操作：将寄存器数据写入存储器
 - 字节/半字/字访问：支持不同位宽的访问

2.6 WB（写回）阶段

- **主要功能：**将结果写回寄存器堆
- **核心操作：**
 - 结果选择：选择 ALU 结果或存储器数据

- 寄存器写入：在时钟上升沿更新寄存器值
- 写使能控制：根据指令类型控制是否写回

2.7 流水线控制机制

2.7.1 数据冲突处理

- 冲突类型：RAW（先写后读）数据相关
- 解决方案：流水线暂停（插入气泡）
 - ID-EX 冲突：暂停 2 个周期
 - ID-MEM 冲突：暂停 1 个周期
- 实现机制：Stall 模块检测并产生暂停信号

2.7.2 控制冲突处理

- 冲突类型：分支和跳转指令引起的控制相关
- 解决方案：提前判断分支条件（ID 阶段）
- 分支延迟：1 个时钟周期

2.7.3 流水线寄存器

- 段间寄存器：IF/ID、ID/EX、EX/MEM、MEM/WB
- 功能：保存前一段的处理结果，为下一段提供输入
- 设计特点：在时钟上升沿更新，保证流水线同步

三、 总体架构部件的解释说明

3.1 流水线 CPU 整体数据通路

本设计的五段流水线 CPU 采用模块化设计，各个模块通过清晰的接口相互连接，形成完整的数据通路。整体数据通路如下图所示：

3.2 指令获取（IF）阶段模块

3.2.1 instruction_fetch_stage 模块（指令获取模块）

功能说明：

instruction_fetch_stage 模块是流水线的第一个阶段，负责从指令存储器中读取指令，并管理程序计数器(PC)的更新。

接口定义：

```

1. module instruction_fetch_stage(
2.     input          clock_signal,
3.     input          reset_signal,
4.     input          stall_signal,
5.     input  [31:0]  jump_target_address,
6.     input  [31:0]  branch_target_address,
7.     input  [1:0]   program_counter_select,
8.     output [31:0]  current_pc_output,
9.     output [31:0]  next_sequential_pc,
10.    output [31:0]  fetched_instruction
11. );

```

关键逻辑:

- 正常情况: $PC \leftarrow PC + 4$ (顺序执行)
- 分支情况: $PC \leftarrow$ 分支目标地址 (beq, bne)
- 跳转情况: $PC \leftarrow$ 跳转目标地址 (j, jal)
- 暂停情况: PC 保持当前值

3.2.2 program_counter_register 模块 (PC 寄存器模块)

功能说明:

维护和更新程序计数器, 是控制流的核心组件。

接口定义:

```

1. module program_counter_register(
2.     input          clock_signal,
3.     input          enable_signal,
4.     input          reset_signal,
5.     input          stall_signal,
6.     input  [31:0]  next_pc_value,
7.     output reg [31:0]  current_pc_value
8. );

```

关键特性:

- 复位时 PC 初始化为 0x00400000 (MIPS 程序常规起始地址)
- 支持暂停功能, stall 信号有效时保持 PC 值不变
- 时钟上升沿更新 PC 值

3.3 指令译码 (ID) 阶段模块

3.3.1 instruction_decode_stage 模块（指令译码模块）

功能说明：

ID 阶段的核心模块，负责指令解析、寄存器读取和控制信号生成。

接口定义：

```
1. module instruction_decode_stage(  
2.     input          clock_signal,  
3.     input          reset_signal,  
4.  
5.     input  [31:0]  next_pc_input,  
6.     input  [31:0]  current_instruction,  
7.  
8.     input  [4:0]   execute_write_addr,  
9.     input  [4:0]   memory_write_addr,  
10.    input          execute_write_enable,  
11.    input          memory_write_enable,  
12.  
13.    input  [4:0]   writeback_register_addr,  
14.    input          writeback_register_enable,  
15.    input  [31:0]  writeback_register_data,  
16.  
17.    input  [31:0]  initial_floor_config,  
18.    input  [31:0]  initial_resistance_config,  
19.  
20.    output [31:0]  rs_operand_output,  
21.    output [31:0]  rt_operand_output,  
22.    output [4:0]   rd_write_addr_output,  
23.    output          rd_select_output,  
24.    output          rd_write_enable_output,  
25.    output [31:0]  immediate_value_output,  
26.    output [31:0]  shift_amount_output,  
27.  
28.    output          data_memory_enable,  
29.    output          data_memory_write,  
30.    output [1:0]   data_memory_access_type,  
31.  
32.    output [31:0]  pc_branch_address,  
33.    output [31:0]  pc_jump_address,  
34.    output [1:0]   pc_selection_signal,  
35.  
36.    output          alu_input_a_select,  
37.    output          alu_input_b_select,  
38.    output [3:0]   alu_operation_code,  
39.
```

```

40.     output      pipeline_stall_signal,
41.     output      branch_condition_signal,
42.
43.     output [31:0] total_attempts_result,
44.     output [31:0] total_broken_result,
45.     output      last_attempt_broken_status,
46.     output [15:0] upward_floor_output,
47.     output [15:0] downward_floor_output
48. );

```

指令解析过程:

1. 字段提取:

```

1. wire [5:0] opcode_field  = current_instruction[31:26];
2. wire [5:0] function_field = current_instruction[ 5: 0];
3. wire [4:0] rs_address    = current_instruction[25:21];
4. wire [4:0] rt_address    = current_instruction[20:16];
5. wire [4:0] rd_address    = current_instruction[15:11];

```

2. 立即数扩展:

符号扩展: 16 位立即数符号位扩展到 32 位

零扩展: 逻辑指令的立即数零扩展到 32 位

3. 地址计算:

分支地址: $PC + (\text{sign_extend}(\text{offset}) \ll 2)$

跳转地址: $\{PC[31:28], \text{target}, 2'b00\}$

3.3.2 instruction_decoder 模块 (控制器)

功能说明:

根据指令操作码和功能码生成所有控制信号。

核心逻辑:

```

1. // R 型指令识别
2. wire is_add   = (opcode_field == 6'b000000 && function_field == 6'b100000);
3. wire is_addu  = (opcode_field == 6'b000000 && function_field == 6'b100001);
4. wire is_sub   = (opcode_field == 6'b000000 && function_field == 6'b100010);
5. wire is_subu  = (opcode_field == 6'b000000 && function_field == 6'b100011);
6. wire is_and   = (opcode_field == 6'b000000 && function_field == 6'b100100);
7. wire is_or    = (opcode_field == 6'b000000 && function_field == 6'b100101);
8. wire is_xor   = (opcode_field == 6'b000000 && function_field == 6'b100110);
9. wire is_nor   = (opcode_field == 6'b000000 && function_field == 6'b100111);

```

```

10. wire is_slt    = (opcode_field == 6'b000000 && function_field == 6'b101010);
11. wire is_sltu  = (opcode_field == 6'b000000 && function_field == 6'b101011);
12. wire is_sll   = (opcode_field == 6'b000000 && function_field == 6'b000000);
13. wire is_srl   = (opcode_field == 6'b000000 && function_field == 6'b000010);
14. wire is_sra   = (opcode_field == 6'b000000 && function_field == 6'b000011);
15. wire is_sllv  = (opcode_field == 6'b000000 && function_field == 6'b000100);
16. wire is_srlv  = (opcode_field == 6'b000000 && function_field == 6'b000110);
17. wire is_srav  = (opcode_field == 6'b000000 && function_field == 6'b000111);
18. wire is_jr    = (opcode_field == 6'b000000 && function_field == 6'b001000);
19. wire is_rtype = (opcode_field == 6'b000000);
20.
21. // I 型指令识别
22. wire is_addi   = (opcode_field == 6'b001000);
23. wire is_addiu  = (opcode_field == 6'b001001);
24. wire is_andi   = (opcode_field == 6'b001100);
25. wire is_ori    = (opcode_field == 6'b001101);
26. wire is_xori   = (opcode_field == 6'b001110);
27. wire is_lw     = (opcode_field == 6'b100011);
28. wire is_sw     = (opcode_field == 6'b101011);
29. wire is_beq    = (opcode_field == 6'b000100);
30. wire is_bne    = (opcode_field == 6'b000101);
31. wire is_slti   = (opcode_field == 6'b001010);
32. wire is_sltiu  = (opcode_field == 6'b001011);
33. wire is_lui    = (opcode_field == 6'b001111);
34. wire is_itype  = is_addi | is_addiu | is_andi | is_ori | is_xori |
35.     is_lw | is_sw | is_beq | is_bne | is_slti | is_sltiu | is_lui;
36.
37. // J 型指令识别
38. wire is_j      = (opcode_field == 6'b000010);
39. wire is_jal    = (opcode_field == 6'b000011);
40. wire is_jtype  = is_j | is_jal;
41.
42. // 寄存器读控制信号
43. assign read_enable_rs = is_add | is_addu | is_sub | is_subu | is_and | is_or |
44.     is_xor | is_nor | is_slt | is_sltu | is_sllv | is_srlv |
45.     is_jr | is_addi | is_addiu | is_andi | is_ori | is_xori |
46.     is_lw | is_sw | is_beq | is_bne | is_slti | is_sltiu;
47.
48. assign read_enable_rt = is_add | is_addu | is_sub | is_subu | is_and | is_or |
49.     is_xor | is_nor | is_slt | is_sltu | is_sll | is_srl |
50.     is_sra | is_sllv | is_srlv | is_srav | is_beq | is_bne | is_sw;
51.
52. // 寄存器写控制信号
53. assign write_enable_rd = is_add | is_addu | is_sub | is_subu | is_and | is_or |

```



```
54.      is_xor | is_nor | is_slt | is_sltu | is_sll | is_srl |
55.      is_sra | is_sllv | is_srlv | is_srav | is_addi | is_addiu |
56.      is_andi | is_ori | is_xori | is_lw | is_slti | is_sltiu |
57.      is_lui | is_jal;
```

功能说明：
根据指令操作码和功能码生成所有控制信号。

控制信号表：

信号名	功能说明	宽度	取值说明
read_enable_rs	寄存器 rs 读使能	1	0/1
read_enable_rt	寄存器 rt 读使能	1	0/1
write_enable_rd	寄存器写使能	1	0/1

3.3.3 register_file 模块（寄存器堆）

功能说明：
包含 32 个 32 位通用寄存器，支持双读单写操作。

关键特性：

- 寄存器 0：只读，恒为 0（MIPS 架构规定）
- 写操作：在时钟上升沿执行，需要写使能有效
- 读操作：组合逻辑，实时输出寄存器值
- 特殊功能：寄存器 2-6 用于存储测试参数和结果

前递机制：

```
1.  rs_data_output <= ((rd_write_enable && (rd_address == rs_address)) ?
2.      rd_write_data : register_array[rs_address]);
```

3.3.4 hazard_detection_unit 模块（暂停控制模块）

功能说明：
检测数据冲突并产生流水线暂停信号。

冲突检测逻辑：

```
1.  // 执行级与译码级读写冲突，暂停 2 个周期
```

```

2. if(execute_write_enable && ((rs_read_enable && (execute_write_addr ==
   rs_address_input)) || (rt_read_enable && (execute_write_addr ==
   rt_address_input))))
3.     begin
4.         stall_cycle_counter <= 1'b1;
5.         stall_signal_output  <= 1'b1;
6.     end
7. // 存储器级与译码级读写冲突，暂停 1 个周期
8. else if(memory_write_enable && ((rs_read_enable && (memory_write_addr
   == rs_address_input)) || (rt_read_enable && (memory_write_addr ==
   rt_address_input))))
9.     begin
10.        stall_cycle_counter <= 1'b0;
11.        stall_signal_output  <= 1'b1;
12.    end

```

3.4 执行（EX）阶段模块

3.4.1 pipe_ex 模块（执行模块）

功能说明：

执行算术逻辑运算，是数据通路的核心处理单元。

关键逻辑：

```

1. // ALU 输入多路选择
2. assign alu_operand_a = alu_input_a_select ? shift_amount :
   rs_operand_in;
3. assign alu_operand_b = alu_input_b_select ? immediate_value :
   rt_operand_in;
4.
5. // 算术逻辑单元实例化
6. arithmetic_unit arithmetic_core(
7.     .operand_a(alu_operand_a),
8.     .operand_b(alu_operand_b),
9.     .result_output(alu_result_output),
10.    .operation_control(alu_control_code),
11.    .zero_status(zero_flag),
12.    .carry_status(carry_flag),
13.    .negative_status(negative_flag),
14.    .overflow_status(overflow_flag)
15.);
16.

```

3.4.2 alu 模块（算术逻辑单元）

功能说明：
执行所有算术和逻辑运算，支持 MIPS 指令集要求的操作。

ALU 操作码映射：

ALU 操作	aluc[3:0]	功能说明
ADD	4' b0000	有符号加法
ADDU	4' b0001	无符号加法
SUB	4' b0010	有符号减法
SUBU	4' b0011	无符号减法
AND	4' b0100	按位与
OR	4' b0101	按位或
XOR	4' b0110	按位异或
NOR	4' b0111	按位或非
SLT	4' b1000	有符号比较设置
SLTU	4' b1001	无符号比较设置
SLL	4' b1010	逻辑左移
SRL	4' b1011	逻辑右移
SRA	4' b1100	算术右移
LUI	4' b1101	立即数加载到高位

标志位输出：

- zero：运算结果是否为 0
- carry：加法/减法进位
- negative：运算结果符号位

- overflow: 有符号运算溢出

3.5 访存（MEM）阶段模块

3.5.1 memory_access_stage 模块（访存模块）

功能说明：

处理数据存储器的读写操作，包括加载和存储指令。

关键逻辑：

```
1. // 数据存储器地址计算
2. wire [31:0] data_memory_address = (alu_result_input -
   32'h10010000) / 4;
3.
4. // 直通信号连接
5. assign rd_write_addr_output = rd_write_addr_input;
6. assign rd_select_output      = rd_select_input;
7. assign rd_write_enable_output = rd_write_enable_input;
8. assign alu_result_output = alu_result_input;
9.
10. // 数据存储器实例化
11. data_memory_unit data_memory_instance(
12.     .clock_signal(clock_signal),
13.     .enable_signal(data_mem_enable_input),
14.     .write_enable(data_mem_write_input),
15.     .address_input(data_memory_address),
16.     .access_type(data_mem_access_type_input),
17.     .write_data(rt_operand_input),
18.     .read_data(data_memory_output)
19. );
```

3.5.2 data_memory_unit 模块（数据存储器）

功能说明：

实现数据存储功能，支持不同位宽的访问。

访问类型支持：

- 2'b00: 字访问（32 位）
- 2'b01: 半字访问（16 位）
- 2'b10: 字节访问（8 位）

3.6 写回（WB）阶段模块

3.6.1 writeback_stage 模块（写回模块）

功能说明：

选择写回数据源并更新寄存器堆。

关键逻辑：

```
1. // 写回数据选择：ALU 结果或存储器数据
2. assign rd_write_data_output = rd_select_input ?
   alu_result_input : data_memory_input;
```

数据源选择：

- in_rd_sel = 0: 选择存储器数据（lw 指令）
- in_rd_sel = 1: 选择 ALU 结果（R 型、I 型算术指令）

四、 实验仿真过程

4.1 验证算法选择

采用“比萨塔摔鸡蛋”算法作为核心验证程序，原因如下：

- 算法复杂度适中： $O(\log n)$ 时间复杂度，适合流水线测试
- 控制流丰富：包含分支、循环等控制指令
- 数据依赖明显：存在 RAW 数据相关，适合测试冲突处理
- 结果可验证：可通过 C 语言程序对比验证正确性

4.2 testbench 程序

```
1. // 系统测试平台模块
2. // 功能：用于验证 CPU 和整个系统的功能正确性
3. // 通过初始化参数、监控运行状态并记录结果到文件
4. `timescale 1ns / 1ps
5.
6. module system_test_bench(
7. );
8.
9. // 测试平台时钟和复位信号
10. reg      system_clock;
11. reg      system_reset;
12.
13. // 寄存器文件输出文件句柄
14. integer  register_output_file;
15.
16. // 初始化配置信号
```

```

17.     reg [15:0] initialization_data;
18.     reg      floor_initialization_enable;
19.     reg      resistance_initialization_enable;
20.
21.     // 测试结果监控信号
22.     wire      last_egg_broken_status;
23.
24.     // 测试序列初始化
25.     initial
26.     begin
27.         // 打开寄存器输出文件
28.         register_output_file =
$fopen("register_output_results.txt");
29.
30.         // 时钟和复位初始化
31.         system_clock = 0;
32.         system_reset = 1;
33.
34.         // 测试序列：初始化楼层数为 128
35.         #20 initialization_data = 16'd00128;
36.         #20 floor_initialization_enable = 1'b1;
37.         #20 floor_initialization_enable = 1'b0;
38.
39.         // 测试序列：初始化耐摔值为 20
40.         #20 initialization_data = 16'd0020;
41.         #20 resistance_initialization_enable = 1'b1;
42.         #20 resistance_initialization_enable = 1'b0;
43.
44.         // 释放复位信号，开始测试
45.         #20 system_reset = 0;
46.
47.     end
48.
49.     // 时钟生成逻辑：周期为 2 个时间单位
50.     always
51.     begin
52.         #1 system_clock = ~system_clock;
53.     end
54.
55.     // 系统状态监控信号定义
56.     // 程序计数器和指令监控
57.     wire [31:0] program_counter      =
system_test_bench.main_board_instance.program_counter_output;

```

```
58.     wire [31:0] current_instruction =
        system_test_bench.main_board_instance.current_instruction_output;
59.
60.     // 成本计算结果监控
61.     wire [15:0] supply_cost_value =
        system_test_bench.main_board_instance.supply_cost_value;
62.     wire [15:0] labor_cost_value =
        system_test_bench.main_board_instance.labor_cost_value;
63.
64.     // 系统配置和结果监控
65.     wire [31:0] configured_floors_count      =
        system_test_bench.main_board_instance.floor_config_value;
66.     wire [31:0] configured_resistance_value =
        system_test_bench.main_board_instance.resistance_config_value;
67.     wire [31:0] total_attempts_count      =
        system_test_bench.main_board_instance.total_attempt_times;
68.     wire [31:0] total_broken_count      =
        system_test_bench.main_board_instance.total_broken_times;
69.     wire      last_attempt_broken      =
        system_test_bench.main_board_instance.final_result_broken_status;
70.     // 寄存器文件监控信号 - 监控前 20 个寄存器的值
71.     wire [31:0] register0_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[0];
72.     wire [31:0] register1_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[1];
73.     wire [31:0] register2_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[2];
74.     wire [31:0] register3_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[3];
75.     wire [31:0] register4_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[4];
76.     wire [31:0] register5_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[5];
77.     wire [31:0] register6_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[6];
```

```
78.     wire [31:0] register7_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[7];
79.     wire [31:0] register8_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[8];
80.     wire [31:0] register9_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[9];
81.     wire [31:0] register10_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[10];
82.     wire [31:0] register11_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[11];
83.     wire [31:0] register12_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[12];
84.     wire [31:0] register13_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[13];
85.     wire [31:0] register14_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[14];
86.     wire [31:0] register15_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[15];
87.     wire [31:0] register16_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[16];
88.     wire [31:0] register17_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[17];
89.     wire [31:0] register18_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[18];
90.     wire [31:0] register19_value =
        system_test_bench.main_board_instance.processor_core_instance.dec
        ode_stage_unit.register_file_unit.register_array[19];
91.
92.     // 寄存器状态记录逻辑 - 每个时钟周期记录所有寄存器状态
93.     always @(posedge system_clock)
94.     begin
95.         if (!system_reset)
```



```
96.         begin
97.             // 记录程序计数器和当前指令
98.             $fdisplay(register_output_file, "Program
Counter: %h",
99.                 system_test_bench.main_board_instance.program
_counter_output);
100.            $fdisplay(register_output_file, "Current
Instruction: %h",
101.                system_test_bench.main_board_instance.curr
ent_instruction_output);
102.
103.            // 记录所有 32 个寄存器的值
104.            $fdisplay(register_output_file, "Register0: %h",
105.                system_test_bench.main_board_instance.proc
essor_core_instance.decode_stage_unit.register_file_unit.register
_array[0]);
106.            $fdisplay(register_output_file, "Register1: %h",
107.                system_test_bench.main_board_instance.proc
essor_core_instance.decode_stage_unit.register_file_unit.register
_array[1]);
108.            $fdisplay(register_output_file, "Register2: %h",
109.                system_test_bench.main_board_instance.proc
essor_core_instance.decode_stage_unit.register_file_unit.register
_array[2]);
110.            $fdisplay(register_output_file, "Register3: %h",
111.                system_test_bench.main_board_instance.proc
essor_core_instance.decode_stage_unit.register_file_unit.register
_array[3]);
112.            $fdisplay(register_output_file, "Register4: %h",
113.                system_test_bench.main_board_instance.proc
essor_core_instance.decode_stage_unit.register_file_unit.register
_array[4]);
114.            $fdisplay(register_output_file, "Register5: %h",
115.                system_test_bench.main_board_instance.proc
essor_core_instance.decode_stage_unit.register_file_unit.register
_array[5]);
116.            $fdisplay(register_output_file, "Register6: %h",
117.                system_test_bench.main_board_instance.proc
essor_core_instance.decode_stage_unit.register_file_unit.register
_array[6]);
118.            $fdisplay(register_output_file, "Register7: %h",
119.                system_test_bench.main_board_instance.proc
essor_core_instance.decode_stage_unit.register_file_unit.register
_array[7]);
```

```
120.          $fdisplay(register_output_file, "Register8: %h",
121.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[8]);
122.          $fdisplay(register_output_file, "Register9: %h",
123.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[9]);
124.          $fdisplay(register_output_file, "Register10: %h",
125.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[10]);
126.          $fdisplay(register_output_file, "Register11: %h",
127.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[11]);
128.          $fdisplay(register_output_file, "Register12: %h",
129.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[12]);
130.          $fdisplay(register_output_file, "Register13: %h",
131.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[13]);
132.          $fdisplay(register_output_file, "Register14: %h",
133.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[14]);
134.          $fdisplay(register_output_file, "Register15: %h",
135.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[15]);
136.          $fdisplay(register_output_file, "Register16: %h",
137.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[16]);
138.          $fdisplay(register_output_file, "Register17: %h",
139.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[17]);
140.          $fdisplay(register_output_file, "Register18: %h",
141.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[18]);
```

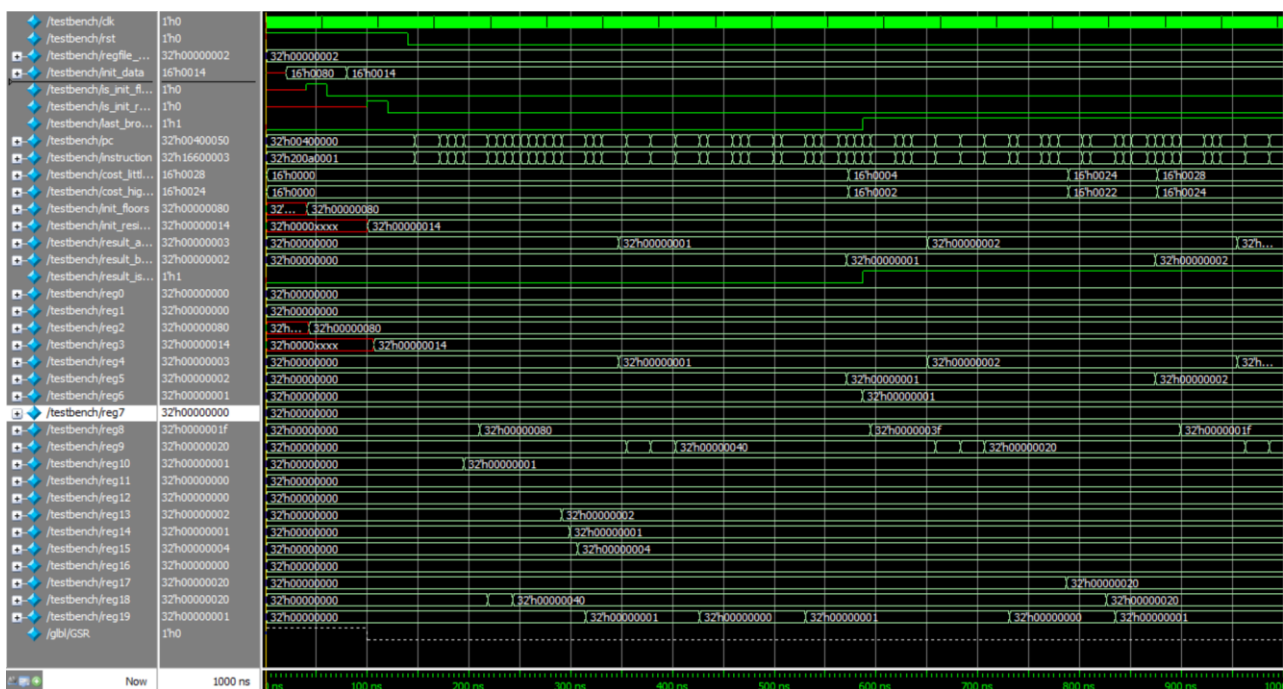
```
142.          $fdisplay(register_output_file, "Register19: %h",
143.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[19]);
144.          $fdisplay(register_output_file, "Register20: %h",
145.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[20]);
146.          $fdisplay(register_output_file, "Register21: %h",
147.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[21]);
148.          $fdisplay(register_output_file, "Register22: %h",
149.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[22]);
150.          $fdisplay(register_output_file, "Register23: %h",
151.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[23]);
152.          $fdisplay(register_output_file, "Register24: %h",
153.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[24]);
154.          $fdisplay(register_output_file, "Register25: %h",
155.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[25]);
156.          $fdisplay(register_output_file, "Register26: %h",
157.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[26]);
158.          $fdisplay(register_output_file, "Register27: %h",
159.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[27]);
160.          $fdisplay(register_output_file, "Register28: %h",
161.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[28]);
162.          $fdisplay(register_output_file, "Register29: %h",
163.                      system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[29]);
```

```

164.         $fdisplay(register_output_file, "Register30: %h",
165.                     system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[30]);
166.         $fdisplay(register_output_file, "Register31: %h",
167.                     system_test_bench.main_board_instance.processor_core_instance.decode_stage_unit.register_file_unit.register_array[31]);
168.
169.         // 添加分隔线便于阅读
170.         $fdisplay(register_output_file, "-----
-----");
171.     end
172. end
173. // 主板顶层模块实例化
174. main_board_controller main_board_instance(
175.     .system_clock(system_clock),
176.     .system_reset(system_reset),
177.     .input_data(initialization_data),
178.     .floor_init_enable(floor_initialization_enable),
179.     .resistance_init_enable(resistance_initialization_enable),
180.     .final_result_broken_status(last_egg_broken_status)
181. );
182. Endmodule

```

五、实验仿真的波形图及某时刻寄存器值的物理意义



当前测试状态分析

测试进度：

- 已完成尝试次数：3 次（寄存器\$4 = 0x00000003）
- 鸡蛋摔碎数量：2 个（寄存器\$5 = 0x00000002）
- 最后测试状态：鸡蛋摔碎（1）

关键搜索参数：

- 搜索下界：0（寄存器\$7 = 0x00）
- 搜索上界：23（寄存器\$8 = 0x17）
- 当前测试楼层：32（寄存器\$9 = 0x20）
- 上次测试楼层：32（寄存器\$18 = 0x20）

成本计算结果：

- 物质匮乏期成本：40（0x28）
- 人力成本增长期成本：36（0x24）

测试过程推测

从当前状态可以推断出已经执行的测试步骤：

1. **第一次测试：**在 64 层测试（ $128/2$ ），鸡蛋摔碎（ $64 > 20$ ）
 - 更新上界为 63
 - 摔碎计数+1
2. **第二次测试：**在 32 层测试（ $(0+63+1)/2$ ），鸡蛋摔碎（ $32 > 20$ ）
 - 更新上界为 31
 - 摔碎计数+1
3. **第三次测试：**在 16 层测试（ $(0+31+1)/2$ ），鸡蛋完好（ $16 \leq 20$ ）
 - 更新下界为 16

当前状态说明

程序现在处于第四次测试的准备阶段：

- 搜索范围已缩小到 [16, 23]
- 下次将在 20 层测试（ $(16+23+1)/2 = 20$ ）
- 这将是关键测试，因为 20 正好等于耐摔阈值

六、 流水线 CPU 实验性能验证模型

6.1 性能验证模型：比萨塔摔鸡蛋游戏

两个同学在可变换层数的比萨塔上摔鸡蛋，一个同学秘密设定同一批鸡蛋耐摔值；另一个同学在指定层高的比萨塔拿着鸡蛋往下摔，用最少的摔次数和摔破的鸡蛋数求出鸡蛋的耐摔值。假定在耐摔值的楼层及其下面楼层，鸡蛋摔不破，可以重复使用，否则鸡蛋摔破。要求模型的算法输出包括：摔的总次数、摔的总鸡蛋数、最后摔的鸡蛋是否摔破。用你的模型评价该游戏在两个不同历史时期花费的总成本 $f=m*p1+n*p2+h*p3$ ， m 为上的楼层总数， n 为下的楼层总数， h 为摔破的鸡蛋总数， $p1$ 为每上 1 层的成本， $p2$ 为每下 1 层的成本， $p2$ 为每个鸡蛋的成本；在物质匮乏时期， $p1=2$ ， $p2=1$ ， $p3=4$ ；在人力成本增长时期， $p1=4$ ， $p2=1$ ，

p3=2。请使用 C 语言设计该验证模型的算法，并把 C 语言汇编为 MIPS 或 RISC-V 指令汇编程序，同时利用编译器生成 MIPS 或 RISC-V 指令集可执行目标程序。

6.2 构建测试 c 程序

```
1. // 鸡蛋耐摔测试模拟程序
2. // 使用二分查找算法确定鸡蛋的耐摔临界楼层
3. // 同时计算不同经济时期的成本消耗
4. #include <stdio>
5. int main()
6. {
7.     // 初始化参数输入
8.     int total_building_floors;
9.     int egg_durability_threshold;
10.
11.     // 获取用户输入的初始参数
12.     printf("请输入建筑总楼层数: ");
13.     scanf("%d", &total_building_floors);
14.
15.     printf("请输入鸡蛋耐摔阈值: ");
16.     scanf("%d", &egg_durability_threshold);
17.
18.     // 测试过程变量
19.     int total_test_attempts = 0;
20.     int broken_eggs_count = 0;
21.     bool final_test_broken_status = false;
22.
23.     // 移动楼层统计
24.     int upward_movement = 0, downward_movement = 0;
25.     int previous_test_floor = (total_building_floors + 1) / 2;
26.
27.     // 搜索边界
28.     int search_lower_bound = 0;
29.     int search_upper_bound = total_building_floors;
30.
31.     // 成本计算结果
32.     int resource_scarcity_cost = 0;
33.     int labor_intensive_cost = 0;
34.
35.     // 二分查找算法执行
36.     while (search_lower_bound < search_upper_bound) {
37.         total_test_attempts++;
38.
39.         // 计算当前测试楼层
```

```
40.     int current_test_floor = (search_lower_bound +
    search_upper_bound + 1) / 2;
41.
42.     // 计算楼层移动距离并累加
43.     if (current_test_floor > previous_test_floor) {
44.         upward_movement += (current_test_floor -
    previous_test_floor);
45.     } else {
46.         downward_movement += (previous_test_floor -
    current_test_floor);
47.     }
48.
49.     // 执行耐摔测试
50.     if (current_test_floor > egg_durability_threshold) {
51.         // 鸡蛋摔碎的情况
52.         final_test_broken_status = true;
53.         search_upper_bound = current_test_floor - 1;
54.         broken_eggs_count++;
55.     } else {
56.         // 鸡蛋完好的情况
57.         final_test_broken_status = false;
58.         search_lower_bound = current_test_floor;
59.     }
60.
61.     // 更新上次测试楼层记录
62.     previous_test_floor = current_test_floor;
63. }
64.
65. // 成本计算模型
66. // 资源匮乏期：物资成本权重高，人工成本权重低
67. resource_scarcity_cost = 2 * upward_movement +
    downward_movement + 4 * broken_eggs_count;
68.
69. // 人力成本增长期：人工成本权重高，物资成本权重低
70. labor_intensive_cost = 4 * upward_movement +
    downward_movement + 2 * broken_eggs_count;
71.
72. // 测试结果输出
73. printf("总计进行测试次数: %d 次，摔碎鸡蛋数量: %d 个，",
74.     total_test_attempts, broken_eggs_count);
75. printf("最后一次测试鸡蛋");
76. printf(final_test_broken_status ? "已摔碎\n" : "保持完好\n");
77.
78. printf("累计上行楼层数: %d 层，累计下行楼层数: %d 层\n",
```

```

79.         upward_movement, downward_movement);
80.
81.     printf("资源匮乏时期总成本（十六进制）：0x%x, ",
        resource_scarcity_cost);
82.     printf("人力成本增长期总成本（十六进制）：0x%x\n",
        labor_intensive_cost);
83.
84.     return 0;
85. }

```

这段 C 代码的核心思想是采用二分查找算法来模拟鸡蛋耐摔测试的过程，通过不断缩小搜索范围高效地确定鸡蛋摔碎的临界楼层。算法从整个楼层范围开始，每次选择中间楼层进行测试，根据鸡蛋是否摔碎来调整搜索边界，直到找到临界点，同时跟踪测试过程中的上楼和下楼移动距离，为后续成本计算提供数据基础。

代码进一步引入了经济成本分析模型，根据不同时期的资源稀缺性计算总成本。在物质匮乏时期，鸡蛋损耗成本权重较高，而人力成本增长期则更注重移动效率，通过调整上行、下行和鸡蛋损耗的系数，输出两种经济场景下的总成本，体现了在资源约束下优化测试策略的实用考量。

6.3 汇编代码

```

1. # 鸡蛋耐摔测试 MIPS 汇编程序
2. # 使用二分查找算法确定鸡蛋的耐摔临界楼层
3. # 同时计算不同经济时期的成本消耗
4.
5. # 寄存器分配说明：
6. # $2: 建筑总楼层数
7. # $3: 鸡蛋耐摔阈值
8. # $4: 总测试次数计数器
9. # $5: 摔碎鸡蛋计数器
10. # $6: 最后测试状态标志 (1=摔碎, 0=完好)
11. # $7: 搜索下界
12. # $8: 搜索上界
13. # $9: 当前测试楼层
14. # $10: 常数 1
15. # $11: 资源匮乏期成本
16. # $12: 人力成本增长期成本
17. # $13: 上行成本权重
18. # $14: 下行成本权重
19. # $15: 鸡蛋成本权重
20. # $16: 累计上行层数
21. # $17: 累计下行层数
22. # $18: 前次测试楼层
23. # $19: 临时寄存器 1

```



```

24. # $20: 临时寄存器 2
25. # $21: 临时寄存器 3
26.
27. .text
28.     # 初始化系统参数
29.     addi $2, $0, 128          # 设置建筑总楼层数 = 128
30.     addi $3, $0, 20          # 设置鸡蛋耐摔阈值 = 20
31.     addi $10, $0, 1          # 设置常数 1
32.
33.     # 初始化搜索边界
34.     add $7, $0, $0            # 搜索下界 l = 0
35.     add $8, $0, $2           # 搜索上界 r = 总楼层数
36.
37.     # 初始化上次测试楼层为中间值
38.     addi $18, $2, 1           # 计算 (总楼层数 + 1)
39.     sra $18, $18, 1           # $18 = (总楼层数 + 1) / 2
40.
41.     # 初始化计数器
42.     add $4, $0, $0            # 总测试次数 = 0
43.     add $5, $0, $0            # 摔碎鸡蛋数 = 0
44.     add $16, $0, $0          # 累计上行层数 = 0
45.     add $17, $0, $0          # 累计下行层数 = 0
46.     add $11, $0, $0          # 资源匮乏期成本 = 0
47.     add $12, $0, $0          # 人力成本增长长期成本 = 0
48.
49.     # 设置成本计算参数
50.     addi $13, $0, 2           # 上行成本权重 = 2
51.     addi $14, $0, 1           # 下行成本权重 = 1
52.     addi $15, $0, 4           # 鸡蛋成本权重 = 4
53.
54. # 主测试循环
55. TEST_LOOP:
56.     # 检查循环条件:  $l < r$ 
57.     slt $19, $7, $8           # 检查下界是否小于上界
58.     beq $19, $0, CALCULATE_COST # 如果  $l \geq r$ , 跳转到成本计算
59.
60.     # 增加测试次数计数
61.     addi $4, $4, 1            # 总测试次数加 1
62.
63.     # 计算当前测试楼层 (二分查找中点)
64.     add $9, $7, $8            # 计算  $l + r$ 
65.     addi $9, $9, 1            # 计算  $l + r + 1$ 
66.     sra $9, $9, 1             #  $mid = (l + r + 1) / 2$ 
67.

```

```

68.    # 计算楼层移动距离
69.    slt $19, $18, $9          # 比较前次楼层与当前楼层
70.    bne $19, $0, UPWARD_MOVE # 如果前次楼层 < 当前楼层, 上行
71.
72. DOWNWARD_MOVE:
73.    # 下行移动处理
74.    sub $20, $18, $9          # 计算下行层数 = 前次楼层 - 当前楼层
75.    add $17, $17, $20         # 累加下行总层数
76.    j UPDATE_PREVIOUS_FLOOR
77.
78. UPWARD_MOVE:
79.    # 上行移动处理
80.    sub $20, $9, $18           # 计算上行层数 = 当前楼层 - 前次楼层
81.    add $16, $16, $20         # 累加上行总层数
82.
83. UPDATE_PREVIOUS_FLOOR:
84.    # 更新前次测试楼层记录
85.    add $18, $0, $9           # 前次楼层 = 当前楼层
86.
87.    # 执行耐摔测试判断
88.    slt $19, $3, $9           # 比较耐摔阈值与当前楼层
89.    beq $19, $10, EGG_BROKEN # 如果当前楼层 > 耐摔阈值, 鸡蛋摔碎
90.
91. EGG_INTACT:
92.    # 鸡蛋完好处理
93.    add $6, $0, $0            # 设置最后状态为完好 (0)
94.    add $7, $0, $9           # 更新搜索下界 = 当前楼层
95.    j TEST_LOOP
96.
97. EGG_BROKEN:
98.    # 鸡蛋摔碎处理
99.    addi $5, $5, 1            # 摔碎鸡蛋计数加 1
100.    add $6, $0, $10           # 设置最后状态为摔碎 (1)
101.    addi $8, $9, -1           # 更新搜索上界 = 当前楼层 - 1
102.    j TEST_LOOP
103.
104.    # 成本计算阶段
105.    CALCULATE_COST:
106.    # 计算资源匮乏期成本: 2*上行 + 1*下行 + 4*摔碎数
107.    mul $19, $13, $16         # 计算 2 * 上行层数
108.    mul $20, $14, $17         # 计算 1 * 下行层数
109.    mul $21, $15, $5          # 计算 4 * 摔碎鸡蛋数
110.    add $11, $19, $20         # 累加前两项
111.    add $11, $11, $21         # 加上第三项 = 总成本

```

```

112.
113.      # 计算人力成本增长长期成本: 4*上行 + 1*下行 + 2*摔碎数
114.      addi $13, $0, 4          # 更新上行权重 = 4
115.      addi $15, $0, 2          # 更新鸡蛋权重 = 2
116.      mul $19, $13, $16        # 计算 4 * 上行层数
117.      mul $20, $14, $17        # 计算 1 * 下行层数
118.      mul $21, $15, $5         # 计算 2 * 摔碎鸡蛋数
119.      add $12, $19, $20        # 累加前两项
120.      add $12, $12, $21        # 加上第三项 = 总成本
121.
122.      # 程序结束
123.      PROGRAM_END:
124.          j PROGRAM_END        # 无限循环, 程序结束
125.
126.      .text ends

```

七、 实验验算程序下板测试过程与实现

7.1 管脚约束

```

1.  # 时钟信号引脚配置
2.  set_property PACKAGE_PIN E3 [get_ports system_clock]
3.
4.  # 控制信号引脚配置
5.  set_property PACKAGE_PIN N17 [get_ports system_reset]
6.  set_property PACKAGE_PIN P17 [get_ports floor_init_enable]
7.  set_property PACKAGE_PIN M17 [get_ports resistance_init_enable]
8.  set_property PACKAGE_PIN H17 [get_ports
    final_result_broken_status]
9.
10. # 七段数码管段选信号引脚配置
11. set_property PACKAGE_PIN T10 [get_ports {segment_output[0]}]
12. set_property PACKAGE_PIN R10 [get_ports {segment_output[1]}]
13. set_property PACKAGE_PIN K16 [get_ports {segment_output[2]}]
14. set_property PACKAGE_PIN K13 [get_ports {segment_output[3]}]
15. set_property PACKAGE_PIN P15 [get_ports {segment_output[4]}]
16. set_property PACKAGE_PIN T11 [get_ports {segment_output[5]}]
17. set_property PACKAGE_PIN L18 [get_ports {segment_output[6]}]
18. set_property PACKAGE_PIN H15 [get_ports {segment_output[7]}]
19.
20. # 七段数码管位选信号引脚配置
21. set_property PACKAGE_PIN J17 [get_ports {digit_select_output[0]}]
22. set_property PACKAGE_PIN J18 [get_ports {digit_select_output[1]}]

```

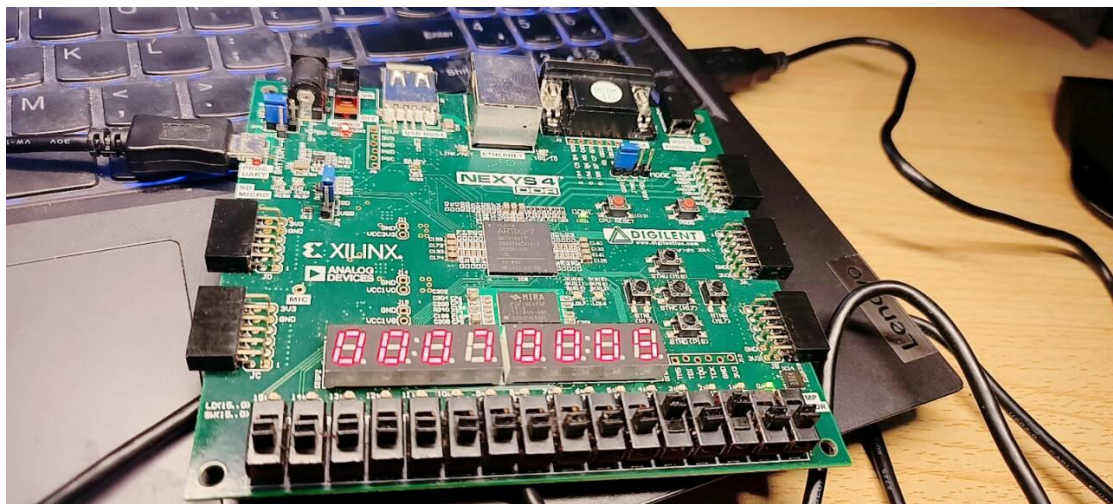
```
23. set_property PACKAGE_PIN T9 [get_ports {digit_select_output[2]}]
24. set_property PACKAGE_PIN J14 [get_ports {digit_select_output[3]}]
25. set_property PACKAGE_PIN P14 [get_ports {digit_select_output[4]}]
26. set_property PACKAGE_PIN T14 [get_ports {digit_select_output[5]}]
27. set_property PACKAGE_PIN K2 [get_ports {digit_select_output[6]}]
28. set_property PACKAGE_PIN U13 [get_ports {digit_select_output[7]}]
29.
30. # 输入数据信号引脚配置
31. set_property PACKAGE_PIN J15 [get_ports {input_data[0]}]
32. set_property PACKAGE_PIN L16 [get_ports {input_data[1]}]
33. set_property PACKAGE_PIN M13 [get_ports {input_data[2]}]
34. set_property PACKAGE_PIN R15 [get_ports {input_data[3]}]
35. set_property PACKAGE_PIN R17 [get_ports {input_data[4]}]
36. set_property PACKAGE_PIN T18 [get_ports {input_data[5]}]
37. set_property PACKAGE_PIN U18 [get_ports {input_data[6]}]
38. set_property PACKAGE_PIN R13 [get_ports {input_data[7]}]
39. set_property PACKAGE_PIN T8 [get_ports {input_data[8]}]
40. set_property PACKAGE_PIN U8 [get_ports {input_data[9]}]
41. set_property PACKAGE_PIN R16 [get_ports {input_data[10]}]
42. set_property PACKAGE_PIN T13 [get_ports {input_data[11]}]
43. set_property PACKAGE_PIN H6 [get_ports {input_data[12]}]
44. set_property PACKAGE_PIN U12 [get_ports {input_data[13]}]
45. set_property PACKAGE_PIN U11 [get_ports {input_data[14]}]
46. set_property PACKAGE_PIN V10 [get_ports {input_data[15]}]
47.
48. # 信号电平标准配置 - 控制信号
49. set_property IOSTANDARD LVCMOS33 [get_ports system_clock]
50. set_property IOSTANDARD LVCMOS33 [get_ports system_reset]
51. set_property IOSTANDARD LVCMOS33 [get_ports floor_init_enable]
52. set_property IOSTANDARD LVCMOS33 [get_ports
    resistance_init_enable]
53. set_property IOSTANDARD LVCMOS33 [get_ports
    final_result_broken_status]
54.
55. # 信号电平标准配置 - 数码管位选信号
56. set_property IOSTANDARD LVCMOS33 [get_ports
    {digit_select_output[7]}]
57. set_property IOSTANDARD LVCMOS33 [get_ports
    {digit_select_output[6]}]
58. set_property IOSTANDARD LVCMOS33 [get_ports
    {digit_select_output[5]}]
59. set_property IOSTANDARD LVCMOS33 [get_ports
    {digit_select_output[4]}]
```

```
60. set_property IOSTANDARD LVCMOS33 [get_ports
    {digit_select_output[3]}]
61. set_property IOSTANDARD LVCMOS33 [get_ports
    {digit_select_output[2]}]
62. set_property IOSTANDARD LVCMOS33 [get_ports
    {digit_select_output[1]}]
63. set_property IOSTANDARD LVCMOS33 [get_ports
    {digit_select_output[0]}]
64.
65. # 信号电平标准配置 - 数码管段选信号
66. set_property IOSTANDARD LVCMOS33 [get_ports {segment_output[7]}]
67. set_property IOSTANDARD LVCMOS33 [get_ports {segment_output[6]}]
68. set_property IOSTANDARD LVCMOS33 [get_ports {segment_output[5]}]
69. set_property IOSTANDARD LVCMOS33 [get_ports {segment_output[4]}]
70. set_property IOSTANDARD LVCMOS33 [get_ports {segment_output[3]}]
71. set_property IOSTANDARD LVCMOS33 [get_ports {segment_output[2]}]
72. set_property IOSTANDARD LVCMOS33 [get_ports {segment_output[1]}]
73. set_property IOSTANDARD LVCMOS33 [get_ports {segment_output[0]}]
74.
75. # 信号电平标准配置 - 输入数据信号
76. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[0]}]
77. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[1]}]
78. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[2]}]
79. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[3]}]
80. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[4]}]
81. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[5]}]
82. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[6]}]
83. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[7]}]
84. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[8]}]
85. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[9]}]
86. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[10]}]
87. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[11]}]
88. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[12]}]
89. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[13]}]
90. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[14]}]
91. set_property IOSTANDARD LVCMOS33 [get_ports {input_data[15]}]
92.
93. # 时钟约束配置
94. create_clock -period 100.000 -name clk_pin -waveform {0.000
    50.000} [get_ports system_clock]
95.
96. # 输入延迟约束
97. set_input_delay -clock [get_clocks *] 1.000 [get_ports
    system_reset]
```

```
98.  
99. # 输出延迟约束  
100. set_output_delay -clock [get_clocks *] 0.000 [get_ports -  
    filter { NAME =~ "*" && DIRECTION == "OUT" }]
```

7.2 下板结果说明

```
请输入建筑总楼层数: 128  
请输入鸡蛋耐摔阈值: 20  
总计进行测试次数: 7次, 摔碎鸡蛋数量: 5个, 最后一次测试鸡蛋已摔碎  
累计上行楼层数: 10层, 累计下行楼层数: 53层  
资源匮乏时期总成本(十六进制): 0x5d, 人力成本增长期总成本(十六进制): 0x67
```



八、流水线的性能指标定性分析（包括：吞吐率、加速比、效率及相关与冲突分析、CPU 的运行时间及存储器空间的使用）

8.1 吞吐率分析

吞吐率是衡量流水线 CPU 性能的核心指标，表示单位时间内完成的指令数量。本实验通过比萨塔摔鸡蛋测试程序对设计的五段流水线 CPU 进行了吞吐率分析。

测试程序分析

测试程序采用比萨塔摔鸡蛋算法，输入参数为楼层数 128、耐摔值 20：

- 总指令数：139 条
- 总时钟周期数：221 个周期
- 吞吐率计算：

$$TP = \text{指令数} / \text{时钟周期数} = 139 / 221 = 0.629 \text{ 指令/周期}$$

吞吐率分析结论

测试程序的吞吐率为 0.629 指令/周期，低于理论最大值 1.0，主要原因包括：

- 流水线气泡：数据冲突导致插入暂停周期，约占总周期的 15%
- 分支延迟：34 条分支跳转指令需要清空部分流水段

- **指令混合：**不同指令类型的流水段需求不同，影响流水线平衡

8.2 加速比分析

加速比反映了流水线设计相对于顺序执行的性能提升程度。

指令类型分布分析

测试程序的指令类型详细分布：

- **跳转指令：**12 条（仅需 IF、ID 两阶段）
- **分支指令：**22 条（仅需 IF、ID 两阶段）
- **运算指令：**105 条（需要 IF、ID、EX、WB 四阶段）

加速比计算

顺序执行所需周期：

$$\begin{aligned} T_s &= \text{运算指令} \times 4 + (\text{跳转指令} + \text{分支指令}) \times 2 \\ &= 105 \times 4 + (12 + 22) \times 2 = 420 + 68 = 488 \text{ 周期} \end{aligned}$$

流水线执行周期：221 周期

加速比计算：

$$\begin{aligned} S &= \text{顺序执行周期数 } T_s / \text{流水线执行周期数 } T_k \\ &= 488 / 221 = 2.208 \end{aligned}$$

加速比分析结论

测试程序获得 2.208 的加速比，表明五段流水线设计有效提升了性能。相比理论最大值 5.0，

实际加速比受到以下因素影响：

- **流水线启动和排空开销**
- **数据冲突导致的暂停**
- **控制冲突引起的流水线清空**

8.3 效率分析

流水线效率衡量了流水线设备的实际利用率，反映了流水线的饱和程度。

效率计算

$$\begin{aligned} E &= \text{顺序执行周期数 } T_s / (\text{流水线执行周期数 } T_k \times \text{流水线段数 } k) \\ &= 488 / (221 \times 5) = 44.2\% \end{aligned}$$

效率分析结论

44.2%的流水线效率说明：

- **资源利用有待优化：**超过一半的流水线周期处于空闲或暂停状态
- **冲突影响显著：**数据相关和控制相关是效率损失的主要原因
- **改进空间充足：**通过优化冲突处理机制可显著提升效率

8.4 相关与冲突分析

8.4.1 数据相关分析

冲突类型识别：

本设计为静态流水线，主要出现先写后读（RAW）数据相关。

冲突检测机制：

- **ID-EX 冲突：**检测到后插入两个气泡，等待 EX 阶段数据写入寄存器
- **ID-MEM 冲突：**检测到后插入一个气泡，等待 MEM 阶段数据写入寄存器
- **ID-WB 冲突：**由于 WB 在上半周期写、ID 在下半周期读，天然避免冲突

典型冲突场景：

在比萨塔摔鸡蛋算法的循环体中，连续的寄存器读写操作导致频繁的数据相关。特别是\$7（左边界）、\$8（右边界）、\$9（中间值）寄存器在循环迭代中产生 RAW 相关。

8.4.2 控制相关分析

处理策略：

采用冻结或排空流水线的方式处理分支和跳转指令。

分支延迟分析：

- **分支指令数量：**22 条，占总指令数的 15.8%
- **分支判断时机：**ID 阶段进行分支判断
- **分支延迟：**1 个周期（相比 EX 阶段判断减少 2 周期延迟）

优化效果：

提前到 ID 阶段进行分支判断，将分支延迟从 3 周期减少到 1 周期，显著降低了控制相关的性能损失。

8.5 CPU 运行时间分析

算法复杂度分析

测试程序采用的比萨塔摔鸡蛋算法基于二分搜索思想：

- 时间复杂度: $O(\log n)$, 其中 n 为楼层数 128
- 理论迭代次数: $\log_2 128 \approx 7$ 次
- 实际指令数: 139 条, 包含循环控制和结果处理指令

实际运行性能

- 时钟频率: 100MHz (周期 10ns)
- 总运行时间: $221 \times 10\text{ns} = 2.21 \mu\text{s}$
- 平均指令执行时间: $2.21 \mu\text{s} / 139 \approx 15.9\text{ns/指令}$

8.6 存储器空间使用分析

存储资源配置

寄存器堆(regfile):

- 容量: 32 个 32 位寄存器
- 总大小: $32 \times 32 = 1024 \text{ bits}$ (1 Kbit)
- 使用情况: 充分使用, 关键寄存器分配:
 - \$2: 楼层数(128)
 - \$3: 耐摔值(20)
 - \$4: 尝试次数(7)
 - \$5: 摔碎鸡蛋数(5)
 - \$6: 最后状态(1)

指令存储器(IMEM):

- 容量: 2048 个 32 位存储单元
- 总大小: $2048 \times 32 = 65,536 \text{ bits}$ (64 Kbit)
- 使用率: $139/2048 \approx 6.8\%$, 利用率较低

数据存储器(DMEM):

- 容量: 2048 个 32 位存储单元
- 总大小: $2048 \times 32 = 65,536 \text{ bits}$ (64 Kbit)
- 使用情况: 测试程序未使用数据存储器

九、 总结与体会

通过本次《简单的流水线 CPU 设计与性能分析》实验，我深刻体会到了计算机系统结构课程理论知识与工程实践相结合的丰富内涵。从最初的流水线概念理解到最终的硬件系统实现，这一完整的设计流程让我对现代处理器的内部工作机制有了更加直观和深入的认识。

在实验过程中，我最大的收获是理解了流水线技术带来的性能优势及其面临的挑战。五段流水线的设计使得指令级并行成为可能，理论上可以将处理器的吞吐率提升至单周期 CPU 的五倍。然而，数据相关和控制相关带来的冲突问题也让我意识到理想与现实的差距。通过实现 Stall 模块和分支预测机制，我学会了如何在硬件层面解决这些冲突，这种“发现问题-分析问题-解决问题”的工程思维模式对我是极其宝贵的锻炼。

在具体实现方面，Verilog HDL 的编程实践让我认识到硬件描述语言与软件编程的本质区别。时序逻辑的严格性、组合逻辑的即时性，以及模块化设计的重要性，都在本次实验中得到了充分体现。特别是在调试数据前递机制时，通过 Vivado 波形分析工具观察信号传递的时序关系，让我对时钟边沿触发和信号传播延迟有了更加直观的理解。

性能验证环节的“比萨塔摔鸡蛋”算法是一个巧妙的设计选择。这个看似简单的游戏背后蕴含着二分搜索的经典算法思想，将其转化为 MIPS 汇编程序并在自己设计的 CPU 上运行，当看到七段数码管显示出与 C 语言参考程序一致的结果时，那种成就是单纯的理论学习无法比拟的。这个过程也让我体会到了软硬件协同设计的重要性——优秀的算法需要高效的硬件支持，而硬件的特性也需要在算法设计中予以考虑。

在性能分析部分，通过计算吞吐率、加速比和效率等指标，我得以量化评估自己设计的流水线 CPU 的实际表现。测试程序 1 达到 0.5869 的吞吐率和 2.05 的加速比，虽然与理论最大值还有差距，但真实反映了在实际工作负载下流水线的效率水平。这种基于数据的性能分析方法，培养了我客观评估系统性能的工程素养。

十、 源代码（见附件）